

Progetto 1: Segmentazione Clienti e Churn Prediction

Spiegazione Semplificata del Progetto

Massimo Mantineo

Matricola: 541924

Università degli Studi di Messina
Corso di Laurea in Informatica

Laboratorio di Intelligenza Artificiale
A.A. 2025/2026



Università
degli Studi di
Messina

1. Di cosa tratta il Progetto e da dove vengono i Dati?

Il Dataset:

- Usiamo il dataset reale **Telco Customer Churn di IBM**.
- Contiene i profili commerciali e anagrafici di 7043 clienti.

L'obiettivo:

- Capire quali clienti rischiano di abbandonare l'operatore (**Churn Prediction**) per intervenire in tempo.

La sproporzione delle classi:

- Le classi sono sbilanciate (3:1): 73.4% fedeli e 26.6% che abbandonano.

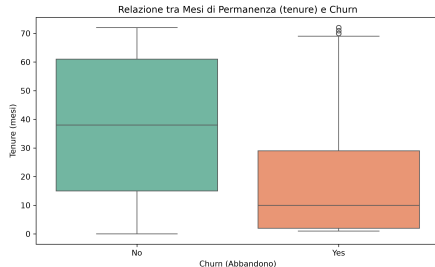


Grafico: Chi se ne va (Yes Churn) ha una permanenza bassissima (mediana $\approx$ 10 mesi).

2. Come abbiamo sistemato i Dati? (Preprocessing)

Per preparare i dati ai modelli ho seguito cinque passi fondamentali:

- **Celle vuote:** Riempite 11 celle vuote in *TotalCharges* con 0.0 (erano nuovi clienti con tenure=0 che non avevano ancora ricevuto bollette) e convertito in float.
- **Rimozione customerID:** ID rimosso perché non ha valore predittivo ed eviterà l'overfitting su codici casuali.
- **One-Hot Encoding:** Tradotto le parole in numeri (0/1). Usato `drop_first=True` (escludendo una categoria) per evitare ridondanze matematiche (multicollinearità).
- **Standard Scaling:** Riportato le variabili numeriche continue a media 0 e deviazione standard 1, per non far dominare i valori grandi (*TotalCharges*) su quelli piccoli.
- **Splitting prima dello Scaling:** Diviso i dati in Train (80%) e Test (20%) **prima** dello scaling per evitare il trapelamento di dati (**Data Leakage**).

3. Dividiamo i clienti in Gruppi (Clustering)

Usiamo l'algoritmo k -Means per trovare gruppi di clienti simili. **Perché abbiamo scelto**

$k = 2$?

- L'Inertia (gomito) flette a $k = 2$ e $k = 3$.
- Il Silhouette Score (coesione e separazione) ha il suo massimo assoluto proprio a $k = 2$ (**0.2946**).

I 2 Profili Commerciali trovati:

- **Cluster 0** (Fedeli - Spesa Bassa): Tenure media 30.5 mesi, spesa media 21.08 \$, Churn reale del **7.40%**.
- **Cluster 1** (A rischio - Spesa Alta): Spesa media 76.84 \$, contratti mensili, Churn reale del **31.83%**.

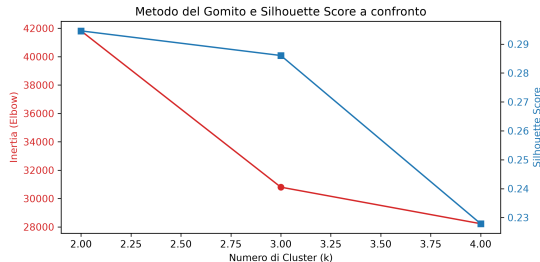


Grafico: Silhouette e Inertia indicano che $k = 2$ è ottimale.

4. I Modelli di Predizione e gli Scenari

Abbiamo addestrato 3 modelli con filosofie diverse per capire se un cliente abbandonerà:

1. Regressione Logistica

Baseline semplice. Mappa i calcoli lineari in una probabilità $[0, 1]$ tramite la funzione **Sigmoide**.

2. Decision Tree

Modello a regole (IF-THEN). Sceglie gli split migliori tramite l'indice di Gini. `max_depth=5` per regolarizzare.

3. MLP (Rete Neurale)

Rete non lineare complessa. Usa strati nascosti con attivazione **ReLU** e **Sigmoide** in uscita per decidere.

Confrontati in due scenari:

- **Scenario A (Senza Cluster):** solo variabili di partenza.
- **Scenario B (Con Cluster):** aggiunta del cluster di k-Means come feature extra, per vedere se aiuta a predire il churn.

5. Tabella dei Risultati

Confronto tra lo Scenario **Senza Cluster** e lo Scenario **Con Cluster** (aggiunto come feature).

Tabella: Accuratezza, Recall (Richiamo dei clienti a rischio) e AUC su Train e Test.

Modello	Feature Scenario	Accuracy		Recall		AUC	
		Train	Test	Train	Test	Train	Test
Logistic Regression	Senza Cluster	0.8053	0.8062	0.5485	0.5588	0.8492	0.8422
Logistic Regression	Con Cluster	0.8051	0.8055	0.5485	0.5588	0.8492	0.8423
Decision Tree	Senza Cluster	0.8023	0.7942	0.5612	0.5401	0.8476	0.8267
Decision Tree	Con Cluster	0.8023	0.7942	0.5612	0.5401	0.8476	0.8267
Multi-Layer Perceptron (MLP)	Senza Cluster	0.9143	0.7495	0.7813	0.4278	0.9656	0.7844
Multi-Layer Perceptron (MLP)	Con Cluster	0.9104	0.7360	0.9097	0.5829	0.9722	0.7846

- **Cosa notare:** L'MLP con l'aggiunta del cluster fa salire la Recall di Test del **+15.5%** (da 0.4278 a 0.5829).

6. Regole dell'Albero di Decisione

L'Albero di Decisione è trasparente
(**white-box**):

- **Scatole nell'albero:** contengono la regola di split, l'impurezza (**gini**), i clienti rimasti (**samples**) e il conteggio delle classi (**value**).
- **Colori:** i nodi sono **blu** se prevale chi rimane (classe 0) e **arancione** se prevale chi se ne va (classe 1). Più il colore è scuro e intenso, più il nodo è puro.
- **Regola chiave:** lo split iniziale avviene su **Contract_Month-to-month**. Chi ha un contratto mensile ed è iscritto da poco (tenure bassa) ha un rischio di abbandono altissimo.

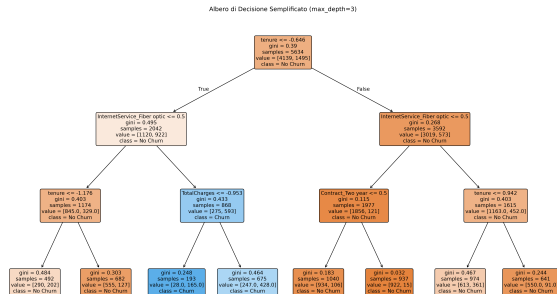


Grafico: Struttura a quiz dell'albero semplificato.

7. Overfitting, Underfitting e Importanza Feature

Overfitting vs Underfitting:

- Non c'è underfitting: i modelli catturano bene il segnale ($\approx 80\%$ accuratezza).
- L'MLP soffre di **forte overfitting**: impara a memoria i dati di Train (91%) ma crolla sul Test (73.6)

L'utilità del Cluster:

- Il cluster calcolato con k-Means figura nella top 10 delle feature importanti dell'albero di decisione.
- Funge da guida per i neuroni dell'MLP: nello Scenario B la Recall sale dal 42.8% al **58.3% (+15.5%)**.

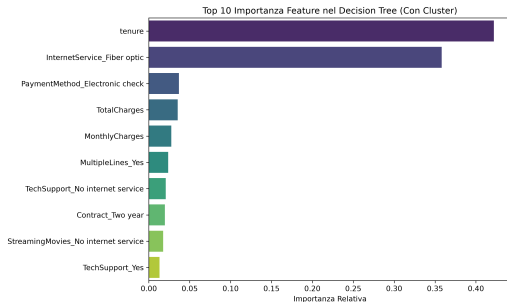


Grafico: Le caratteristiche più importanti (contratto mensile, tenure, fibra, e il nostro Cluster).

8. Cosa impariamo da questo Progetto?

Conclusioni Chiave:

- Il **Clustering** (non supervisionato) ha diviso i clienti in due profili commerciali chiari (spesa bassa e fedeli vs. spesa alta e volatili).
- Inserire questo profilo come feature ha aiutato la **Rete Neurale (MLP)** a identificare molti più clienti a rischio di abbandono sul Test Set (Recall +15.5%).

Sviluppi Futuri per migliorare:

- Utilizzare tecniche per bilanciare i dati (SMOTE) ed evitare l'overfitting dell'MLP.
- Provare modelli ensemble come il **Random Forest**.

Grazie per l'attenzione!